Lab 3 — Linux Commands

Lab materials: http://user.ceng.metu.edu.tr/~ceng111/lab

1. Running Executables

To execute a compiled binary or executable you must specify its full path or use the `./` prefix.

```
./hello.exe          # run the executable in the current directory
/full/path/progName  # also works with absolute or relative path
./scripts/setup.sh   # Execute a shell script located in the scripts folder
```

2. `wget` — Downloading Files from the Internet

The `wget` command downloads files from the internet directly to your local system. It's useful for downloading files non-interactively in scripts.

Usage: `wget [options] url`

```
wget https://ceng.metu.edu.tr   # downloads the page as index.html
```

Options

- `-O filename` — save the downloaded file under a different name (`--output-document`):

  ```
  wget -O logo.png https://webmail.ceng.metu.edu.tr/images/ceng-logo.png
  ```

- `-P directory` — save the file to the specified directory (`--directory-prefix`):

  ```
  wget -P ~/Documents https://webmail.ceng.metu.edu.tr/images/ceng-logo.png
  ```

3. Wildcards

Lab material: http://user.ceng.metu.edu.tr/~ceng111/lab/wildcards.zip

```
wget http://user.ceng.metu.edu.tr/~ceng111/lab/wildcards.zip
unzip wildcards.zip
```

`*` — Zero or More Characters

```
ls *         # list everything, including contents of directories
ls al*        # files and directories starting with 'al'
ls *A        # everything ending with 'A'
ls w*A       # starting with 'w' and ending with 'A'
ls -d w*A    # -d lists files and only the directory names (not its contents)
ls -d a*e*f     # starts with 'a', contains 'e', ends with 'f'
```

! `ls *` lists not just files but also the *contents* of directories so, the directory names and their files are both shown. You can use -d option to suppress opening the directory contents.

`?` — Exactly One Character

`?` matches **exactly one** character; unlike `*`, it cannot match zero characters.

```
ls ?         # files whose name is exactly 1 character
ls ??*       # files with at least two characters
```

```
ls ?esktop   # one character followed by "esktop"
ls D*esktop  # matches Desktop since * can be zero characters
ls De*k?op   # combined use of * and ?
ls ?*        # everything with one or more characters
```

Nested Directory Example

```
cp */*/*.avi ./videos/   # copy .avi files into the videos folder
rm */*/*.avi             # delete all .avi files in subdirectories
```

The first `*` matches a top-level directory, the second `*` a subdirectory, and `*.avi` matches files with that extension.

Filenames Containing Wildcard Characters

If a filename contains `*` or `?`, use an escape character:

```
touch ceng\*111   # creates a file named ceng*111
rm ceng\*111      # deletes it the same way
```

4. `find` — Searching for Files

`find` prints the paths of all files and directories under the given location — it works recursively.

```
find .              # scan recursively from the current directory
find ~              # scan the home directory
find ../lab2        # show everything inside '../lab2'
find . -name index.html          # search by name (case-sensitive)
find -iname InDex.html           # case-insensitive search
find . | grep html               # find any file with 'html' in its name
find . -name "*.html"            # wildcard matching -- quotes are required!
find . -iname "*ceng???"         # case-insensitive with wildcard
find . -name "file?.txt"         # Match files such as file1.txt, fileA.txt, etc.
find -type f -empty              # find empty files
find -type d -empty              # find empty directories
find / -size +100M               # Find files larger than 100MB across the system
find /tmp -user luna             # Find files owned by luna in /tmp
```

! Wildcard quoting rule: When using wildcards with `-name` or `-iname`, the pattern **must be quoted** (`"*.html"`). Without quotes, the shell expands the pattern itself before `find` sees it, so `find` never receives a wildcard and may return unexpected results.

5. `file` — Identifying File Types

The `file` command provides information about a file's type, which is helpful for unknown or non-standard files.

```
file filename                  # Display type information for filename
file data.tar.gz               # Show type and compression details of data.tar.gz
file /bin/ls                   # Check the type of a binary executable
```

6. `history` — Command History

Displays a list of previously entered commands in the terminal.

```
history                          # List all history
history 20                       # Show the last 20 commands
history 20 | grep wget           # Filter history to show only 'wget' commands
```

7. `diff` — Comparing Files

`diff` compares two files line by line and shows the differences between them. It is ideal for comparing two versions of code.

Basic Usage

```
diff file1.txt file2.txt         # Compare two text files
diff -c file1.txt file2.txt      # Context mode, shows surrounding lines for
    differences
diff -u file1.txt file2.txt      # Unified mode, commonly used for code diffs
diff -y file1.txt file2.txt      # Shows the diffs in side-by-side format
```

8. `tar` — Archive Files

`tar` is used to compress or decompress collections of files and directories into a single archive file.

Lab material: wget https://user.ceng.metu.edu.tr/~ceng111/lab/hello.tar.gz

```
tar -xvf hello.tar.gz            # Extract a .tar archive in the current directory
tar -cvf hey.tar.gz hello.c hello.txt  # Compress files into a hey.tar.gz archive
tar -xvf hey.tar.gz -C ~/Desktop       # Extract to a specific directory
tar -tvf hey.tar.gz              # List contents of an archive without extracting
```

9. `ssh`, `scp`, `sftp` — Remote Machine Access

`ssh` — Connecting to a Remote Machine

`ssh` allows secure, encrypted connections to remote systems, often used for remote command execution or system administration.
To connect ineks outside the school: First, connect to VPN using the following link: ttps://faq.cc.metu.edu.tr/tr/groups/vpn-servisi

```
ssh exxxxxxx@external.ceng.metu.edu.tr -p 8085 # connect to external from outside
ssh inekX                                      # then connect to an inekX
```

! Actions you perform and files you save while connected via **`ssh` stay on that remote machine**, they do not transfer to your local computer.

`scp` — Copying Files

`scp` is used for secure file transfers between local and remote systems, or be- tween two remote systems.
`scp [source] [destination]` is the general format. You need to know the remote file path; if unsure, connect via `ssh` first and run `pwd`.

```
scp -P 8085 file.txt e7digit@external.ceng.metu.edu.tr:/home/bs0?/e7digit/Desktop/
scp localfile.txt user@remote:/path/    # Copy file from local to remote server
scp user@remote:/path/file.txt ./       # Copy file from remote to local directory
scp -r /local/dir user@remote:/remote/dir    # Recursively copy directory to
    remote
```

`sftp` — Interactive File Transfer

`sftp` is an interactive file transfer protocol that operates over SSH, allowing secure file management on remote systems.
`sftp` opens a prompt; commands typed at this prompt run **on the remote machine**. To run a command on your *local* machine, prefix it with `l` (for *local*).

```
sftp -P 8085 e7digit@external.ceng.metu.edu.tr   # connect then a prompt opens
put localfile.txt                  # Upload a local file to the remote system
get remotefile.txt                 # Download file from remote to local
mkdir /path/newdir                 # Create directory on remote system
lls                                # List files on the local system
```

Commands used at the prompt:

Command	Description
`ls` / `lls`	list remote / local directory
`pwd` / `lpwd`	show remote / local working directory
`cd` / `lcd`	change remote / local directory
`get filename`	copy file from remote to local directory
`put filename`	copy local file to remote directory

! Do not confuse `get` and `put`! If you accidentally `put` a file instead of `get`ting one that exists on the remote, the remote file will be overwritten with an empty file and permanently lost.

10. `nano` — Text Editor

`nano <filename>` opens the file in nano. If the file does not exist, it is created.

Interface

In the shortcut notation, `^` means **Ctrl** and `M` means **Alt**.

Shortcut Reference

Full shortcut list: https://www.nano-editor.org/dist/latest/cheatsheet.html

To display line numbers while editing: `nano -l <file>`

Shortcut	Function
`Alt+N`	Turn line numbers on/off
`Ctrl+/`	go to a specific line
`Ctrl+A`	go to beginning of line
`Ctrl+E`	go to end of line
`Ctrl+->` (`Ctrl+Space`)	move one word forward
`Ctrl+<-` (`Alt+Space`)	move one word backward
`Ctrl+W`	search (exit search with Ctrl+X)
`Alt+U`	undo
`Ctrl+K`	cut current line (into clipboard)
`Alt+6`	copy current line into clipboard (without deleting)
`Ctrl+U`	paste; press again to paste another copy
`Ctrl+6`	set mark / start selection
`Backspace / Del`	delete a character
`Alt+Del`	delete the current line
`Ctrl+S`	save (stays in editor)
`Ctrl+O`	save as (*write out*)
`Ctrl+X`	close buffer, exit from nano

Cutting and Pasting Lines

To cut a line, place the cursor on it and press `Ctrl+K`. The line is copied to the buffer and removed from the file. To paste the clipboard content, press `Ctrl+U`. Each `Ctrl+U` pastes one copy. To cut multiple lines, press `Ctrl+K` repeatedly; each cut appends to the buffer.

✓ The clipboard is **cleared** when you move the cursor between cuts. To accumulate multiple lines in the clipboard, press `Ctrl+K` back-to-back without moving the cursor in between.

Selecting Text to Cut

Start a selection with `Ctrl+6`, then move the cursor (arrow keys or `Ctrl+->`, etc.). The selection is highlighted. Press `Ctrl+K` to cut it.

✓ If you accidentally changed something and do not want to save: `Ctrl+X` → `N` → Enter.

11. Homework-Style Exercises — nano

1. Deleting N Characters from a Specific Line

Scenario: Delete 3 characters from line 956 of `alice.txt`.

```
nano alice.txt    # open the file
Ctrl+/            # open the go-to-line dialog
956 → Enter      # jump to line 956
Ctrl+A            # go to beginning of line
Del x 3           # press Delete 3 times; each press deletes one character
Ctrl+S            # save
Ctrl+X            # exit
```

✓ To delete many characters at once: start a selection with `Ctrl+6` at the beginning of the line, advance N words with `Ctrl+->` or use the arrow keys, then cut the selection with `Ctrl+K`.

2. Cutting N Lines and Appending Them M Times at End of File

Scenario: Cut 7 lines starting at line 15 of `flatland.txt` and paste them once at the end.

```
nano flatland.txt   # open the file
Ctrl+/              # go to line
15 → Enter          # jump to line 15
Ctrl+K x 7          # press Ctrl+K 7 times; each cut appends a line to buffer
Ctrl+/ Ctrl+V       # go to end of file
Ctrl+U x 1          # paste once: inserts all 7 lines at once
Ctrl+S              # save
Ctrl+X              # exit
```

3. Cutting N Lines to Delete Them (Without Pasting)

Scenario: Permanently delete 5 lines starting at line 30 of `story.txt`.

```
nano story.txt    # open the file
Ctrl+/            # go to line
30 → Enter        # jump to line 30
Ctrl+K x 5        # press Ctrl+K 5 times; lines go to clipboard and are removed
Ctrl+S            # save (without pasting)
Ctrl+X            # exit
```

4. Cutting N Words from a Line and Appending Them M Times at End of File

Scenario: Cut the first 11 words from line 14 of `advice.txt` and paste them 3 times at the end.

```
nano advice.txt    # open the file
Ctrl+/             # go to line
14 → Enter         # jump to line 14
Ctrl+A             # go to beginning of line
Ctrl+6             # start selection
Ctrl+→ x 11        # press Ctrl+Right 11 times; each jump moves one word forward
Ctrl+K             # cut the selected 11 words
Ctrl+/ Ctrl+V      # go to end of file
Ctrl+U x 3         # paste 3 times
Ctrl+S             # save
Ctrl+X             # exit
```